

Documentacion Tecnica delCodigo

Fase 6 - Despliegue - Entregable 4

Proyecto Prototipo academico funcional - Curso IA en Salud
Autor Juan Camilo Garcia Braham
Programa Maestria en IA y Ciencia de Datos
Institucion Universidad Tecnologica de Pereira (UTP)
Marco CRISP-DM/S - SEMMA - DAMA - MLOps
Anno 2026

Tabla de Contenido

Seccion	Contenido
1	Resumen de arquitectura del sistema
2	Descripcion de modulos — API publica
3	Diagrama de dependencias entre modulos
4	Convenciones de codigo y nomenclatura
5	Registro consolidado de decisiones de diseno (D_Fx)
6	Instrucciones para reproducir el entorno de desarrollo
7	Instrucciones para ejecutar las pruebas unitarias y de integracion
8	Limitaciones conocidas y trabajo futuro

1. Resumen de Arquitectura del Sistema

El simulador sigue una arquitectura de capas desacopladas: la capa de generacion de datos (F3), la capa de logica del dominio (F4-A sistema experto), la capa de prediccion (F4-B modelo predictivo), la capa de evaluacion (F5 motor de simulacion) y la capa de presentacion (F6 interfaz web). Cada capa solo depende de las capas inferiores, nunca en sentido contrario.

1.1 Capas del sistema

Capa	Modulo(s)	Responsabilidad	Fase
Generacion de datos	generador_pacientes.py	Genera pacientes sinteticos con distribuciones probabilisticas calibradas. Expone generar_llegadas_tick().	F3
Logica del dominio	sistema_experto.py	Entidades (Paciente, Cama, Area, Piso, Hospital), motor de tick con 19 reglas en 6 capas, modos automatico y asistido.	F4-A
Prediccion	modelo_final_f4b.pkl	Pipeline sklearn StandardScaler + LinearRegression. Predice O en T+4 a partir de 15 features del tick actual.	F4-B
Motor de simulacion	motor_simulacion.py	Orquesta el loop tick a tick. Gestiona estado, lags, prediccion y snapshot del hospital para la UI.	F6
Presentacion	app.py	Interfaz Streamlit. Tres paginas: configuracion, simulador en curso, resultados. Sin logica de dominio propia.	F6

1.2 Principios de diseno aplicados

Separacion de responsabilidades. app.py no contiene logica de dominio. motor_simulacion.py no contiene codigo de UI. sistema_experto.py no sabe que existe una interfaz web.

Inmutabilidad de modulos base. sistema_experto.py, generador_pacientes.py y modelo_final_f4b.pkl no fueron modificados en F6. La integracion ocurre exclusivamente en motor_simulacion.py.

Reproducibilidad garantizada. Toda aleatoriedad esta controlada por semillas documentadas. SEED=99 (evaluacion, D_F5_003), SEED=42 (entrenamiento, D_F3_001). Con la misma semilla y configuracion, la simulacion produce los mismos resultados en cualquier entorno.

Estado centralizado. El estado completo de la simulacion vive en EstadoSimulacion (dataclass). Streamlit lo serializa en st.session_state. No hay estado global mutable fuera de este objeto.

Atomicidad en mutaciones. Toda mutacion que involucra mas de una entidad ocurre dentro de funciones atomicas (_asignar_cama, _trasladar_paciente, etc.). Esto previene estados inconsistentes ante excepciones (D_F4A_001).

2. Descripcion de Modulos - API Publica

2.1 generador_pacientes.py

Genera el flujo de llegada de pacientes sinteticos en cada tick de la simulacion. Las distribuciones de edad, prioridad clinica, area requerida y tiempo de estancia fueron calibradas en F3 con referencia a la normativa colombiana (Resolucion 5596/2015).

Funcion / Constante	Firma	Descripcion
LAMBDA_POR_ESCENARIO	dict[str, float]	Tasa de llegadas por escenario: normal=1.5, alta_demanda=3.0, crisis=5.0 pacientes/tick.
generar_llegadas_tick()	generar_llegadas_tick(tick, escenario, rng) -> list[Paciente]	Genera los pacientes que llegan en el tick indicado. El numero de llegadas sigue una distribucion Poisson(lambda). Cada paciente tiene id unico, edad, prioridad_clinica, area_requerida y tiempo_estancia_esperado asignados segun distribuciones empiricas.

2.2 sistema_experto.py

Modulo central del dominio. Contiene todas las entidades, el motor de tick y las 19 reglas organizadas en 6 capas (elegibilidad, priorizacion, asignacion, sobreocupacion, escalamiento, alta/limpieza).

Clase / Funcion	Tipo	Descripcion
Paciente	dataclass	Entidad paciente. Campos clave: id_paciente, prioridad_clinica (P1-P4), area_requerida, tiempo_espera, tiempo_en_sistema, estado, tick_ingreso.
Cama	dataclass	Entidad cama. Campos: id_cama, tipo, area_id, estado (libre/ocupada/en_limpieza), paciente_id, es_temporal.
Area	dataclass	Entidad area. Incluye capacidad_total, capacidad_disponible, todas_las_camras (oficiales + temporales), prioridades_aceptadas, acepta_desborde.
Hospital	class	Estado global. Indices O(1) por id para pacientes, camas y areas. Fabrica: crear_hospital_referencia(). Metodo: pacientes_esperando().
ResultadoTick	dataclass	Resultado de procesar_tick(). Contiene acciones_ejecutadas, acciones_pendientes, indicador_I, nivel_I, componentes O/E/P/C y alertas.
SistemaExperto	class	Motor de tick. Metodos publicos: procesar_tick(tick, nuevos_pacientes, modo_asistido=False) y confirmar_acciones(seleccion, tick).
calcular_indicador_I()	funcion	Calcula $I = 0.4*O + 0.2*E + 0.2*P + 0.2*C$ sobre el estado actual del hospital. Retorna dict con I, nivel y componentes.

2.3 motor_simulacion.py

Adapta el motor de F5 para operar tick a tick en lugar de en bloque. Introduce ConfigHospital (D_F6_008) para parametrizar la capacidad de cada area desde la UI.

Clase / Funcion	Tipo	Descripcion
ConfigHospital	dataclass	Capacidades configurables: uci=10, urgencias=20, hospitalizacion=40, observacion=15, sala_espera=10. Valores por defecto = D008/F2.

Clase / Funcion	Tipo	Descripcion
EstadoSimulacion	dataclass	Estado completo y reanudable de una simulacion. Contiene hospital, se, rng_sim, rng_gen, tick_actual, historial, historial_O, acciones_pendientes y flags activa/pausada/finalizada.
RegistroTick	dataclass	Una fila del historial. Campos: tick, O_t, E_t, P_t, C_t, I_t, nivel_I, llegadas_t, altas_t, traslados_t, cola_t, n_alertas, pred_O_t4.
crear_estado()	funcion	crear_estado(escenario, modo_asistido, seed, ticks_total, config) -> EstadoSimulacion. Construye el hospital, inicializa SE y generadores RNG. No ejecuta ningun tick.
avanzar_tick()	funcion	avanzar_tick(estado, modelo_pred) -> (ResultadoTick None, list). Procesa exactamente un tick: genera llegadas, llama a procesar_tick(), calcula prediccion y registra en historial.
confirmar_acciones_pendientes()	funcion	confirmar_acciones_pendientes(estado, indices) -> int. Confirma las acciones de modo asistido seleccionadas por el gestor. Retorna el numero ejecutadas.
calcular_resumen()	funcion	calcular_resumen(estado) -> dict. Metricas de regimen estable (ticks >= WARM_UP_TICKS): O_media, I_media, nivel_modal, RMSE pred., cumple_O_rango, cumple_I_nivel.
snapshot_hospital()	funcion	snapshot_hospital(estado) -> dict. Vista serializable del estado actual del hospital para renderizar el grid HTML. Incluye areas, camas, pacientes y metricas del indicador.

2.4 app.py

Punto de entrada de la aplicacion Streamlit. Implementa el router de paginas y los componentes de UI. No contiene logica de dominio.

Funcion	Descripcion
main()	Router principal. Lee st.session_state['pagina'] y despacha a pagina_config(), pagina_simular() o pagina_resultados().
pagina_config()	Pantalla de configuracion. Renderiza controles de infraestructura y parametros. Al pulsar Iniciar crea EstadoSimulacion y navega a pagina_simular().
pagina_simular(modelo)	Pantalla principal. Avanza un tick por rerun, renderiza el grid HTML interactivo, las graficas de evolucion temporal y el panel de modo asistido cuando hay acciones pendientes.
pagina_resultados()	Pantalla de resumen final. Invoca pagina_resumen_inline().
pagina_resumen_inline(sim)	Renderiza metricas de regimen estable, verificacion CE-B y figura completa de O(t) e I(t) con los 200 ticks.
_html_grid(snap, tick_actual, ticks_total)	Genera el HTML del grid de camas con panel JS de ficha de paciente (D_F6_003/007). Incluye leyenda, metricas, barra de progreso, areas con celdas de color y cola de espera.
_fig_evolucion(historial, mostrar_todo)	Genera la figura Matplotlib de O(t) vs prediccion T+4 e I(t) con bandas de nivel. mostrar_todo=False muestra los ultimos 60 ticks; True muestra la serie completa.
cargar_modelo()	Cargado con @st.cache_resource. Deserializa modelo_final_f4b.pkl una sola vez por sesion de Streamlit.

3. Diagrama de Dependencias entre Modulos

El grafo de dependencias es un DAG (grafo aciclico dirigido). Las flechas indican 'importa de':

```
app.py
|-- motor_simulacion.py
| |-- sistema_experto.py (Paciente, Cama, Area, Piso,
| | Hospital, SistemaExperto,
| | calcular_indicador_I)
| |-- generador_pacientes.py (generar_llegadas_tick,
| | LAMBDA_POR_ESCENARIO)
| |-- numpy (RNG, arrays de features)
| `-- [modelo_final_f4b.pkl] (cargado via joblib)
|-- streamlit (UI, session_state)
|-- matplotlib (figuras O(t) e I(t))
`-- joblib (carga del modelo.pkl)

pruebas_integracion_f6.py
|-- motor_simulacion.py (mismas dependencias)
`-- reportlab (PDF de resultados)

generar_manual_usuario_f6.py
|-- reportlab (PDF del manual)
`-- matplotlib (figuras ilustrativas)

generar_documentacion_tecnica_f6.py
`-- reportlab (este documento)
```

Ninguno de los modulos base (sistema_experto.py, generador_pacientes.py) importa modulos de F6. La dependencia es estrictamente descendente.

Modulo	Depende de	No depende de
sistema_experto.py	Python stdlib, dataclasses, numpy	Streamlit, matplotlib, sklearn, motor_simulacion, app
generador_pacientes.py	Python stdlib, numpy, sistema_experto (Paciente)	Streamlit, matplotlib, sklearn, motor_simulacion, app
motor_simulacion.py	sistema_experto, generador_pacientes, numpy, joblib	Streamlit, matplotlib, reportlab, app
app.py	motor_simulacion, streamlit, matplotlib, joblib	reportlab, pruebas_integracion_f6
modelo_final_f4b.pkl	sklearn 1.8, joblib 1.5 (para deserializar)	Cualquier modulo del proyecto

4. Convenciones de Codigo y Nomenclatura

Convencion	Descripcion	Ejemplo
Nombres de variables en espanol	Todos los nombres de variables, funciones y clases del dominio estan en espanol para consistencia con el modelo conceptual de F2.	tiempo_estancia_esperado, prioridad_clinica, calcular_indicador_I
Nombres de modulos en espanol con guion bajo	Los archivos .py del proyecto usan espanol sin tildes.	motor_simulacion.py, sistema_experto.py, generador_pacientes.py
IDs de decisiones D_Fx_NNN	Cada decision de diseno relevante lleva un ID unico con el formato D__. Se referencia en comentarios y en la documentacion formal.	D_F4A_001, D_F5_008, D_F6_002
IDs de riesgos R0N	Los riesgos activos se rastrean con IDs R02, R03, R09 desde F1. Se mencionan en comentarios junto a la regla o codigo que los mitiga.	# R02 — caso borde # R09 — sesgo demografico
Semillas documentadas	Todo uso de aleatoriedad incluye un comentario con el ID de la decision que fija la semilla.	np.random.default_rng(seed) # D_F5_003 SEED=99
Dataclasses inmutables para entidades	Las entidades del dominio son dataclasses. Los campos de identidad (id_*) son strings UUID generados en __post_init__. La mutacion ocurre solo a traves de funciones atomicas.	@dataclass class Paciente: id_paciente: str
Type hints en API publica	Todas las funciones publicas de motor_simulacion.py llevan anotaciones de tipo. Las funciones internas de sistema_experto.py siguen el mismo patron.	def crear_estado(escenario: str, ...) -> EstadoSimulacion
Prefijo _ para funciones internas	Las funciones de uso interno dentro de un modulo llevan prefijo de guion bajo y no forman parte de la API publica.	_procesar_altas(), _asignar_cama(), _html_grid()

5. Registro Consolidado de Decisiones de Diseno

Se listan las decisiones de diseno con impacto arquitectonico o de reproducibilidad acumuladas a lo largo de las seis fases. Las marcadas con Rev.=F6 fueron tomadas en la fase actual.

ID	Decision	Justificacion	Rev.
D_F3_001	SEED global = 42	Semilla base del proyecto para generacion de datos y entrenamiento del modelo.	—
D008	Hospital referencia: UCI=10, Urg=20, Hosp=40, Obs=15, Sala=10	Configuracion base de 95 camas. Parametrizable en F6 via ConfigHospital (D_F6_008).	F6
D_F4A_001	Mutaciones solo en funciones atomicas	Garantiza invariante cama<->paciente. Previene R02.	—
D_F4A_002	TransicionInvalidaError para esperando->dado_de_alta	F2 Tabla 5 declara esta transicion invalida. Detecta bugs de logica en tiempo de ejecucion.	—
D_F4A_003	Orden tick: limpieza-altas-traslados-asignaciones-escalamiento	Modela el ciclo de desinfeccion: camas liberadas en T no estan disponibles en el mismo T.	F5
D_F4A_005	P1 sin UCI permanece en espera, no se reasigna	Preserva componente C del indicador I. RSO-04 activa RES-04 al superar t_umbrales_critico.	F5
D_F4A_006/007	Camas temporales en Area.camadas temporales (lista separada)	No alteran capacidad_total ni el calculo de O. Se destruyen al alta del paciente.	F5
D_F4B_001	Variable objetivo: O_{t+4} , no I	Interpretabilidad clinica maxima. Evita multicolinealidad de predecir I con sus propios componentes como features.	F5
D_F4B_002	Horizonte T+4 (1 hora)	Minimo clinicamente accionable con bajo ruido dado lambda fija.	F5
D_F4B_010	Parsimonia: RL si diff RMSE < 1 pp vs RF	Diff = 0.14 pp -> Regresion Lineal ganadora por navaja de Occam.	—
D_F5_003	SEED evaluacion = 99	Distinta a semillas de entrenamiento (42/123/456/789). Evita data leakage entre entrenamiento y evaluacion.	—
D_F5_005	Gestor asistido confirma 100% en E3	Escenario pesimista conservador para analisis de F6.	F6
D_F5_008	Umbral CE-B ajustados al equilibrio sintetico	O_{eq} calculada via Ley de Little ($L=\lambda \cdot W$). Normal: [35,55]%, Alta demanda: [60,88]%, Crisis: [80,100]%.	F6
D_F5_009	Riesgo R03 declarado MITIGADO en F5	RMSE eval coherente con F4-B. 3 escenarios diferenciables. CE-B cumplido en los tres.	—
D_F6_001	Stack F6: streamlit 1.45.1	Unica dependencia nueva en F6. API session_state estable.	—
D_F6_002	Motor tick a tick con estado en EstadoSimulacion	Permite que Streamlit re-renderice entre ticks sin reiniciar el estado del hospital.	—
D_F6_003	Grid HTML con panel JS de ficha de paciente	Click en cama -> panel lateral con datos del paciente sin salir del componente HTML.	—
D_F6_006	Animacion via st.rerun() completo por tick	Mas simple que st.empty(). Compatible con velocidad 1x-10x. Delay = 0.35/velocidad segundos.	—

ID	Decision	Justificacion	Rev.
D_F6_007	Info paciente: panel lateral JS dentro del grid HTML	Click en cama roja abre ficha; click en libre la cierra.	—
D_F6_008	Pantalla de configuracion separada antes del simulador	Permite configurar infraestructura hospitalaria antes de iniciar. Mas claro para usuarios sin conocimiento tecnico.	—
D_F6_009	Configuracion de pisos/areas dinamica declarada trabajo futuro	Requeriria refactorizar reglas RSO-02 del SE. Fuera del alcance del PMV (TF-01).	—
D_F6_010	Tolerancias pruebas: O+/-2pp, I+/-3pp, RMSE+/-0.5pp	Variabilidad admisible entre motor independiente (F5) y motor integrado (F6) con la misma semilla.	—
D_F6_012	Pruebas de integracion usan hospital D008	Comparabilidad directa con valores de referencia de F5.	—

6. Instrucciones para Reproducir el Entorno de Desarrollo

6.1 Requisitos del sistema

Componente	Version	Notas
Python	3.10+	Probado con Python 3.12
pip / uv	reciente	uv pip fue el gestor usado en desarrollo
SO	cualquiera	Windows 10+, macOS 12+, Ubuntu 22.04+
RAM	2 GB min.	4 GB recomendados
Disco	50 MB	Para los archivos del proyecto

6.2 Estructura de carpetas

```
simulador_f6/  
|- app.py # Punto de entrada Streamlit  
|- motor_simulacion.py # Motor tick a tick (F6)  
|- sistema_experto.py # Motor del dominio (F4-A)  
|- generador_pacientes.py # Generador sintetico (F3)  
|- modelo_final_f4b.pkl # Modelo predictivo (F4-B)  
|- pruebas_integracion_f6.py # Suite de pruebas (F6)  
|- generar_manual_usuario_f6.py # Genera manual PDF (F6)  
|- generar_documentacion_tecnica_f6.py # Este documento
```

6.3 Instalacion de dependencias (entorno limpio)

```
# Opcion A: pip  
python -m venv venv  
source venv/bin/activate # Windows: venv\Scripts\activate  
pip install streamlit==1.45.1 numpy==2.4 pandas==3.0 \  
matplotlib==3.10 seaborn==0.13 \  
scikit-learn==1.8 joblib==1.5 reportlab==4.4  
  
# Opcion B: uv pip (mas rapido)  
uv venv  
source .venv/bin/activate  
uv pip install streamlit==1.45.1 numpy==2.4 pandas==3.0 \  
matplotlib==3.10 seaborn==0.13 \  
scikit-learn==1.8 joblib==1.5 reportlab==4.4
```

6.4 Ejecucion de la aplicacion

```
# Desde la carpeta simulador_f6/  
streamlit run app.py  
  
# La app se abre en http://localhost:8501  
# Para cambiar el puerto:  
streamlit run app.py --server.port 8502
```

6.5 Verificacion del entorno

Para verificar que todas las dependencias estan correctamente instaladas antes de lanzar la app, ejecuta:

```
python -c "  
import streamlit, numpy, pandas, matplotlib, sklearn, joblib  
import sistema_experto, generador_pacientes, motor_simulacion  
import joblib; joblib.load('modelo_final_f4b.pkl')  
print('Entorno OK')  
"
```

7. Instrucciones para Ejecutar las Pruebas

7.1 Pruebas unitarias del sistema experto (F4-A)

La suite de 33 pruebas unitarias cubre cada regla de las 6 capas del sistema experto. No requiere pytest — el runner esta integrado en el propio modulo.

```
# Desde la carpeta del proyecto
python test_sistema_experto.py

# Resultado esperado:
# Total: 33 | Pasadas: 33 | Fallidas: 0
```

7.2 Pruebas de integracion (F6)

Ejecuta los tres escenarios con SEED=99 sobre el prototipo integrado y compara contra valores de referencia de F5. Genera pruebas_integracion_f6.pdf con los resultados.

```
python pruebas_integracion_f6.py

# Resultado esperado:
# RESULTADO GLOBAL: 18/18 checks PASS
# PDF generado: pruebas_integracion_f6.pdf
```

7.3 Verificacion de reproducibilidad

Para verificar que una simulacion es reproducible con la misma semilla, ejecuta el siguiente fragmento:

```
import joblib
from motor_simulacion import crear_estado, avanzar_tick, calcular_resumen

modelo = joblib.load('modelo_final_f4b.pkl')
resultados = []
for _ in range(2): # dos ejecuciones con la misma semilla
    estado = crear_estado('normal', seed=99, ticks_total=200)
    for _ in range(200):
        avanzar_tick(estado, modelo)
        r = calcular_resumen(estado)
        resultados.append(round(r['O_media'], 3))

assert resultados[0] == resultados[1], 'No reproducible!'
print(f'Reproducible: O_media = {resultados[0]}% en ambas ejecuciones')
```

7.4 Regeneracion de documentos PDF de F6

Los tres documentos PDF de F6 pueden regenerarse en cualquier momento ejecutando los scripts correspondientes:

```
python pruebas_integracion_f6.py # Entregable 2
python generar_manual_usuario_f6.py # Entregable 3
python generar_documentacion_tecnica_f6.py # Entregable 4
```

8. Limitaciones Conocidas y Trabajo Futuro

8.1 Limitaciones del prototipo actual

ID	Limitacion	Impacto	Categoria
LIM-01	Datos 100% sinteticos	Los resultados no pueden extrapolarse directamente a hospitales reales sin calibracion con datos historicos reales.	Datos
LIM-02	Lambda fija por escenario durante toda la simulacion	En un hospital real la tasa de llegadas varia por hora del dia, dia de la semana y estacionalidad.	Modelo
LIM-03	Topologia de pisos y areas fija (D008)	No es posible agregar pisos o areas nuevas sin refactorizar las reglas RSO-02 del sistema experto (TF-01).	Arquitectura
LIM-04	Un solo usuario concurrente	Streamlit en modo local no esta disenado para multiples usuarios simultaneos. Cada instancia tiene su propio estado.	Despliegue
LIM-05	Modelo predictivo lineal (RMSE ~2.6 pp)	Adecuado para el PMV academico. En produccion se explorariam modelos con mayor capacidad para capturar no linealidades.	ML
LIM-06	Sin persistencia entre sesiones	Al cerrar el navegador o refrescar la pagina, el estado de la simulacion se pierde. No hay guardado/carga de sesiones.	Despliegue
LIM-07	Sin autenticacion ni control de acceso	Apropiado para uso academico local. Un despliegue real requeriria autenticacion de usuarios.	Seguridad

8.2 Trabajo futuro (TF)

ID	Mejora propuesta	Prerequisito tecnico	Prioridad
TF-01	Configuracion dinamica de pisos y areas	Refactorizar reglas RSO-02 para que la cadena de traslados sea generica sobre cualquier topologia de areas.	Media
TF-02	Lambda variable por hora del dia (patron diurno/nocturno)	Extender generador_pacientes.py con una funcion de intensidad temporal lambda(t).	Alta
TF-03	Calibracion con datos historicos reales anonimizados	Acceso a datos de hospitales colombianos bajo Resolucion 3100/2019. Ajuste de distribuciones en F3.	Alta
TF-04	Guardado y carga de sesiones de simulacion	Serializar EstadoSimulacion a JSON/pickle y restaurarlo desde la UI de Streamlit.	Media
TF-05	Modelo predictivo no lineal (Random Forest o Gradient Boosting)	Dataset mayor (>10.000 filas) con variabilidad de lambda. Ajuste de hiperparametros con CV.	Baja
TF-06	Despliegue productivo multi-usuario	Separar estado de sesion en base de datos (Redis/PostgreSQL). Autenticacion con OAuth. Contenedor Docker.	Alta (v2.0)
TF-07	Integracion con HIS/EHR reales	API de interoperabilidad HL7 FHIR. Cumplimiento de Resolucion 866/2021 (historia clinica digital).	Alta (v2.0)